



Architecture Decision Record (ADR)

เสนอ

อาจารย์ ธนิต เกตุแก้ว

จัดทำโดย

นายพัชรพล สืบทายาท รหัสประจำตัวนักศึกษา 67543210041-9

นายณัฐสิทธิ์ มะโนชัย รหัสประจำตัวนักศึกษา 67543210056-7

นายณัฐวิโรจน์ สุทธิธารมมงคล รหัสประจำตัวนักศึกษา 67543210026-0

นายณัฐพงศ์ จินะปัญญา รหัสประจำตัวนักศึกษา 67543210008-8

เอกสารฉบับนี้เป็นส่วนหนึ่งของรายวิชา ภาคเรียนที่ 2 ปี การศึกษา 2568

มหาวิทยาลัยเทคโนโลยีราชมงคลล้านนา เชียงใหม่

ADR-001: Selection of Modular Monolith Architecture with WebSocket Gateway

Date: 2026-01-05

Status: Accepted

Deciders: ทีมพัฒนา Task Board System

Context

Background

โครงการ Task Board System ถูกพัฒนาขึ้นเพื่อรองรับการทำงานร่วมกันของผู้ใช้หลายคนแบบ Real-time โดยมีฟีเจอร์หลัก เช่น การจัดการ Board, Task, ผู้ใช้ และการแจ้งเตือน ระบบต้องรองรับผู้ใช้จำนวนมาก ตอบสนองได้รวดเร็ว และสามารถพัฒนาให้เสร็จภายในระยะเวลาที่กำหนด

Problem Statement

ทีมต้องเลือกสถาปัตยกรรมซอฟต์แวร์ที่เหมาะสมระหว่างความสามารถในการขยายระบบ (Scalability) และความง่ายในการพัฒนา (Development Speed) ภายใต้ข้อจำกัดด้านเวลา ทีมและงบประมาณ โดยยังคงต้องตอบโจทย์ด้าน Performance, Availability และ Security

Key Drivers

- Functional:
 - รองรับการสร้างและจัดการ Board และ Task
 - รองรับการทำงานร่วมกันแบบ Real-time (Task Update, Notification)
- Quality Attributes:
 - **Performance:** Response time < 500ms สำหรับ 95% ของ requests
 - **Availability:** ระบบพร้อมใช้งานสูงและไม่ล่มทั้งระบบเมื่อเกิดปัญหาบางส่วน
 - **Security:** รองรับ OAuth 2.0, JWT และ Role-Based Access Control
- Constraints:

- ทีมพัฒนา 5 คน และระยะเวลาเพียง 3 เดือน
- งบประมาณจำกัด และต้องใช้ JavaScript/TypeScript

Decision

We will use **Modular Monolith Architecture with WebSocket Gateway**.

Components

- **Frontend:** ส่วนติดต่อผู้ใช้สำหรับจัดการ Board และ Task
- **Backend (Monolith):** ระบบหลักที่แบ่งเป็น Module ชัดเจน (Auth, Board, Task, Notification)
- **WebSocket Gateway:** รองรับ Real-time collaboration และ Fast Task Update
- **Database:** จัดเก็บข้อมูลผู้ใช้, Board และ Task
- **Cache:** ลดภาระการเข้าถึงฐานข้อมูลและเพิ่มความเร็ว
- **Reverse Proxy:** จัดการ routing และ security เบื้องต้น

Technologies

- Frontend: React + TypeScript
- Backend: Node.js + NestJS (Modular structure)
- Database: PostgreSQL
- Cache: Redis
- Real-time: WebSocket / Socket.IO
- Others: Nginx, Object Storage (S3/GCS), Docker

Architectural Patterns

- Modular Monolith
- Layered Architecture (Controller → Service → Repository)
- Role-Based Access Control (RBAC)
- Publish/Subscribe (in-process)

Rationale

Why this decision?

1. เหมาะสมกับ Constraint ของโครงการ

Modular Monolith ช่วยให้ทีมขนาดเล็กสามารถพัฒนาได้รวดเร็วและควบคุมความซับซ้อนได้ดีภายในเวลา 3 เดือน

2. ให้ Performance เพียงพอกับความต้องการ

การใช้ Redis cache และ WebSocket ช่วยให้ Fast Task Update และ Real-time collaboration ทำได้ภายในเกณฑ์ที่กำหนด

3. ลดความเสี่ยงด้าน Cost และ DevOps Complexity

ไม่จำเป็นต้องใช้เครื่องมือ DevOps ที่ซับซ้อนเหมือน Microservices

4. รองรับการเติบโตในอนาคต

สามารถ refactor แยกบาง Module ออกเป็น Microservices ได้ใน Phase ถัดไป

Alternatives Considered

1. Microservices-based Task Board System

- Pros: Scale ได้ดี แยก service ชัดเจน รองรับการเติบโต
- Cons: ระบบซับซ้อน ใช้ทรัพยากรสูง และต้องมี DevOps ขั้นสูง
- Why not chosen: ไม่เหมาะกับทีมขนาดเล็กและเวลาจำกัด

2. Traditional Monolithic Architecture

- Pros: โครงสร้างง่าย พัฒนาเร็ว
- Cons: โค้ดผูกกันแน่น และดูแลยากเมื่อระบบโต
- Why not chosen: ขาดความยืดหยุ่นเมื่อเทียบกับ Modular Monolith

Consequences

Positive (ข้อดี)

-  พัฒนาและ Deploy ได้รวดเร็ว
-  Debug และดูแลรักษาง่าย
-  ต้นทุนต่ำและควบคุมงบประมาณได้

Negative (ข้อเสีย)

-  Scale ได้ทั้งก่อน → **Mitigation:** Deploy หลาย instance และใช้ Load Balancer
-  หากระบบโตมากอาจ refactor ยาก → **Mitigation:** ออกแบบ module boundary ให้ชัดเจนตั้งแต่ต้น

Risks

-  Real-time load สูงเกินคาด
 - Impact: Medium
 - Probability: Medium
 - Mitigation: แยก WebSocket Gateway และเพิ่ม instance ตาม load

Trade-offs

- Simplicity vs Scalability
- Development Speed vs Long-term Flexibility

Compliance

Constraints Met

-  Time & Team Size: Modular Monolith ช่วยให้พัฒนาได้ทันเวลา
-  Budget: ลดค่าใช้จ่ายด้าน Infrastructure และ DevOps

Quality Attributes Addressed

-  Performance: Redis caching และ WebSocket ลด latency
-  Availability: Deploy หลาย instance และ isolate module ภายใน
-  Security: OAuth 2.0, JWT และ RBAC ผ่าน middleware/guard

Notes

Assumptions

- ปริมาณผู้ใช้ในช่วงแรกยังไม่เกินขีดจำกัดของ Monolith
- ทีมมีความคุ้นเคยกับ Node.js และ TypeScript

Future Considerations

- แยก Notification หรือ Analytics ออกเป็น Microservice เมื่อ load เพิ่มขึ้น
- เพิ่ม Monitoring และ Logging เชิงลึก (Prometheus, Grafana)

References

- Software Architecture in Practice – Bass, Clements, Kazman
- ADD-Lite Methodology (ENGSE207 Lecture Notes)